

```
[root@ipa ~]# objdump -p /usr/lib64/libpthread-2.17.so | grep SONAME
SONAME
libpthread.so.0
[root@ipa ~]# █
```

Figure 1: Information about the library

```
[root@ipa ~]# ls -l /usr/lib64/libpthread*
-rwxr-xr-x. 1 root root 141968 Jan 3 21:11 /usr/lib64/libpthread-2.17.so
lrwxrwxrwx. 1 root root 18 Feb 11 13:39 /usr/lib64/libpthread.so.0 -> libpthread-2.17.so
[root@ipa ~]# █
```

Figure 2: The shared library symbolic link is pointing to the *DT_SONAME* field

The shared library has a symbolic link that points to it, which matches its *DT_SONAME* field, as shown in Figure 2.

When an application executes, the runtime linker identifies the required shared libraries and maps them into the program's memory space. On Red Hat Enterprise Linux 7, */lib64/ld-linux-x86-64.so.2* default, which is the 64-bit version of the runtime linker, is provided by the *glibc.x86_64* package. The 32-bit version of the *glibc* library provides the 32-bit version of the runtime linker when it is installed.

The runtime linker uses the *DT_SONAME* names embedded in the application to determine which versions of the libraries to use. It then searches the system for those libraries, using the following steps:

- It searches the directories specified in the *LD_LIBRARY_PATH* environment variable. This step is ignored for *setUID* and *setGID* applications. This environment variable is often used when developing or testing an application, where development libraries are to be used.
- Next, the */etc/ld.so.cache* file is consulted. It contains a compiled list of candidate libraries previously found. The *ldconfig -p* command prints the list of libraries mapped to the */etc/ld.so.cache*. You can refer to Figure 3 for the same.

```
[root@ipa ~]# ldconfig -p
403 libs found in cache '/etc/ld.so.cache'
p11-kit-trust.so (libc6,x86-64) => /lib64/p11-kit-trust.so
libz.so.1 (libc6,x86-64) => /lib64/libz.so.1
libxtables.so.10 (libc6,x86-64) => /lib64/libxtables.so.10
libxslt.so.1 (libc6,x86-64) => /lib64/libxslt.so.1
libxml2.so.2 (libc6,x86-64) => /lib64/libxml2.so.2
libxmlrpc_util.so.3 (libc6,x86-64) => /lib64/libxmlrpc_util.so.3
libxmlrpc_server.cgi.so.3 (libc6,x86-64) => /lib64/libxmlrpc_server.cgi.so.3
libxmlrpc_server_abyss.so.3 (libc6,x86-64) => /lib64/libxmlrpc_server_abyss.so.3
libxmlrpc_server.so.3 (libc6,x86-64) => /lib64/libxmlrpc_server.so.3
libxmlrpc_client.so.3 (libc6,x86-64) => /lib64/libxmlrpc_client.so.3
libxmlrpc_abyss.so.3 (libc6,x86-64) => /lib64/libxmlrpc_abyss.so.3
libxmlrpc.so.3 (libc6,x86-64) => /lib64/libxmlrpc.so.3
libxcb.so.1 (libc6,x86-64) => /lib64/libxcb.so.1
libxcb-xvnc.so.0 (libc6,x86-64) => /lib64/libxcb-xvnc.so.0
libxcb-xv.so.0 (libc6,x86-64) => /lib64/libxcb-xv.so.0
libxcb-xtest.so.0 (libc6,x86-64) => /lib64/libxcb-xtest.so.0
libxcb-xselinux.so.0 (libc6,x86-64) => /lib64/libxcb-xselinux.so.0
libxcb-xkb.so.1 (libc6,x86-64) => /lib64/libxcb-xkb.so.1
libxcb-xinput.so.0 (libc6,x86-64) => /lib64/libxcb-xinput.so.0
libxcb-xinerama.so.0 (libc6,x86-64) => /lib64/libxcb-xinerama.so.0
libxcb-xf86dri.so.0 (libc6,x86-64) => /lib64/libxcb-xf86dri.so.0
libxcb-xfixes.so.0 (libc6,x86-64) => /lib64/libxcb-xfixes.so.0
libxcb-xevie.so.0 (libc6,x86-64) => /lib64/libxcb-xevie.so.0
libxcb-sync.so.1 (libc6,x86-64) => /lib64/libxcb-sync.so.1
libxcb-shm.so.0 (libc6,x86-64) => /lib64/libxcb-shm.so.0
libxcb-shape.so.0 (libc6,x86-64) => /lib64/libxcb-shape.so.0
libxcb-screensaver.so.0 (libc6,x86-64) => /lib64/libxcb-screensaver.so.0
libxcb-res.so.0 (libc6,x86-64) => /lib64/libxcb-res.so.0
libxcb-render.so.0 (libc6,x86-64) => /lib64/libxcb-render.so.0
libxcb-record.so.0 (libc6,x86-64) => /lib64/libxcb-record.so.0
libxcb-randr.so.0 (libc6,x86-64) => /lib64/libxcb-randr.so.0
libxcb-present.so.0 (libc6,x86-64) => /lib64/libxcb-present.so.0
libxcb-glx.so.0 (libc6,x86-64) => /lib64/libxcb-glx.so.0
libxcb-dri3.so.0 (libc6,x86-64) => /lib64/libxcb-dri3.so.0
libxcb-dri2.so.0 (libc6,x86-64) => /lib64/libxcb-dri2.so.0
```

Figure 3: List of libraries mapped in */etc/ld.so.cache* is shown

```
[root@ipa ~]# strings /lib64/ld-linux-x86-64.so.2 | grep '^/'
/-509
/var/tmp
/var/profile
/lib64/
/usr/lib64/
/etc/suid-debug
/%x %s
/proc/self/exe
/etc/ld.so.cache
/proc/sys/kernel/osrelease
/dev/full
/dev/null
/etc/ld.so.preload
[root@ipa ~]# █
```

Figure 4: The library paths that are being referred

- Finally, it searches the directories in the default library paths. The 64-bit shared library loader points to the 64-bit versions of the default location — that is */lib64*, then */usr/lib64*, then */lib*, and then */usr/lib* (Figure 4).

The *ldd* command displays the shared libraries that are required by the specified application at runtime. Each library *DT_SONAME* name is displayed, followed by the path name to the library on the system, as shown in Figure 5.

```
[root@ipa ~]# ldd /usr/sbin/httpd
linux-vdso.so.1 => (0x00007ffc2e5e8000)
libpcre.so.1 => /lib64/libpcre.so.1 (0x00007f0d63f20000)
libselinux.so.1 => /lib64/libselinux.so.1 (0x00007f0d63cf9000)
libaprutil-1.so.0 => /lib64/libaprutil-1.so.0 (0x00007f0d63ad0000)
libcrypt.so.1 => /lib64/libcrypt.so.1 (0x00007f0d63899000)
libexpat.so.1 => /lib64/libexpat.so.1 (0x00007f0d6366f000)
libdb-5.3.so => /lib64/libdb-5.3.so (0x00007f0d632b1000)
libapr-1.so.0 => /lib64/libapr-1.so.0 (0x00007f0d63082000)
libpthread.so.0 => /lib64/libpthread.so.0 (0x00007f0d62e66000)
libdl.so.2 => /lib64/libdl.so.2 (0x00007f0d62c62000)
libc.so.6 => /lib64/libc.so.6 (0x00007f0d62895000)
/lib64/ld-linux-x86-64.so.2 (0x00007f0d64403000)
libuuid.so.1 => /lib64/libuuid.so.1 (0x00007f0d62690000)
libfreebl3.so => /lib64/libfreebl3.so (0x00007f0d6248d000)
[root@ipa ~]# █
```

Figure 5: *ldd* output of *httpd*

Note: The absolute path name of a library is embedded in a program when the shared library does not have the *DT_SONAME* field. In the previous example (refer to Figure 5), the */lib64/ld-linux-x86-64.so.2* references point to the runtime linker itself.

Troubleshooting library dependency issues

Library dependency problems can occur on Red Hat Enterprise Linux systems when some third party software is installed without using *yum* or *RPM*. Also, using the *--force* option or *--nodeps* option with *rpm* can cause library dependency issues, which are easy to identify. When an application references a shared library that is not available, the runtime linker displays a distinctive error message. It displays the name of the first library it cannot find, and then exits the application with an exit status of 127.

When installing some third party application on RHEL without *yum* or with the *--nodeps* or *--force* option, you can check whether the libraries are available or not by executing the *ldd* command as follows:

```
# ldd /usr/bin/application
```

You will get a message like *xyz >> library not found*. So, from this output, the missing libraries can be identified. Once these are identified, they need to be installed on the